

Pythonの手引

2025 年度

京都 Tech

目次

1	文とコメント	1
2	変数宣言	1
3	データ型	1
3.1	データ型	1
4	演算子	2
4.1	算術演算子	2
4.2	論理演算子	3
4.3	比較演算子	3
5	入出力	4
5.1	入力：input()	4
5.2	出力：print()	4
6	制御文	6
6.1	分岐処理：if 文	6
6.2	繰り返し処理	6
6.2.1	while 文	6
6.2.2	for 文	7
6.2.3	continue 文と break 文	8
7	関数：def 文	8
8	組み込み関数とモジュール	9
9	グローバル変数及びローカル変数	10
10	文字列	11
10.1	文字列の宣言	11
10.2	文字列の連結	12

10.3	文字列の参照	12
10.4	文字列の長さ：len()	12
10.5	文字列の切り出し	12
10.6	文字列の分割：split()	13
10.7	文字列の置き換え及び削除：replace()	13
10.8	文字列の検索：find()	13
11	リスト	13
11.1	リストの宣言	14
11.2	リストの要素の参照及び変更	14
11.3	リストの長さ	14
11.4	リストへの要素の追加	15
11.5	リストにある要素が含まれるかの確認	15
11.6	リストの各要素への繰り返し処理	16
12	辞書	16
12.1	辞書の宣言	16
12.2	辞書の要素の参照及び変更	16
12.3	辞書への要素の追加	17
12.4	辞書の長さ	17
12.5	辞書の要素の削除	17
12.6	辞書にキーが含まれるかの確認	17
12.7	辞書の各要素への繰り返し処理	18
13	ファイル入出力	18
13.1	ファイルのオープン及びクローズ	18
13.2	テキストファイルの読み込み	19
13.3	テキストファイルの書き込み	21
14	練習問題	21
15	オブジェクト指向プログラミング	23
15.1	クラス定義とインスタンス生成	23
15.2	クラスの継承およびオーバーライド	25

演習 【1】	27
------------------	----

1 文とコメント

Python の文には文の区切り記号はなく、改行が文の区切りとなる。文を複数行に分けて記述したい場合には、行末にバックスラッシュ (\) を書く。また、1 行内では #以降がコメントとみなされる。

```
x = 1+2 \
    +3      # 行が継続される
print(x)    # x の値を表示、6 が表示される
```

2 変数宣言

Python では変数宣言は行わない。変数名には以下のルールがある。

- (1) 半角のアルファベット、_ (アンダースコア)、数字を組み合わせた変数名とする
- (2) 変数名の先頭は数字は使えない
- (3) 予約語は使えない（予約後の例：and as assert break class continue def del elif else except exec finally for from global if import in is lambda not or pass print raise return try while with yield）

また、アルファベットの大文字と小文字は区別される。

3 データ型

3.1 データ型

Python では変数に値を代入した時にそのデータ型になる。表 3.1 に Python で定義されている主なデータ型を示す。

表 3.1 Python で使用する主なデータ型

データ型	例
bool (真偽)	True、False
str (文字列)	"abc"、'ABC'
int (整数)	123、15
float (浮動小数点)	3.14、123.5 1.23e2 ($=1.23 \times 10^2$)
list (配列)	[12, 'abc', 5]
dictionary (辞書)	{'key1': 'val1', 'key2': 'val2'}

数値から文字列に変換するには `str()` を用いる。また、文字列から数値に変換するには `int()` または `float()` を用いる。

```
num1 = 123
str1 = str(num1)          # str1 には文字列 "123"が代入される

num2 = 123.45
str2 = str(num2)          # str2 には文字列 "123.45"が代入される

str3 = "321"
num3 = int(str3)          # num3 には数値 321 が代入される

str4 = "321.45"
num4 = float(str4)        # num4 には数値 321.45 が代入される
```

4 演算子

4.1 算術演算子

算術演算子を表 4.1 に示す。

表 4.1 Python で使用する算術演算子

演算子	説明	例	例の結果 (x の値)
+	加算	x=10+5	15
-	減算	x=10-5	5
*	乗算	x=10*5	50
/	除算	x=9/2	4.5
//	整数除算	x=9/2	4
%	剰余算	x=10%3	1
**	べき乗	x=2**3	8

なお、 $x = x + a$ は $x += a$ ともかける。

4.2 論理演算子

論理演算子を表 4.2 に示す。

表 4.2 Python で使用する論理演算子

演算子	説明	例 (x=3、y=4 とする)	例の結果
and	論理積	x==3 and y==4	true
or	論理和	x==3 or y==3	true
not	否定	not(x==3)	false

4.3 比較演算子

比較演算子を表 4.3 に示す。

表 4.3 python で使用する比較演算子

演算子	説明	例 (x=3 とする)	例の結果
==	等しい	x == 3	true
!=	等しくない	x != 3	false
>	左辺が大きい	x > 2	true
<	左辺が小さい	x < 2	false
>=	左辺が大きいか等しい	x >= 2	true
<=	左辺が小さいか等しい	x <= 2	false

5 入出力

5.1 入力：input()

Python でキーボードからデータを入力するには input() 関数を使用する。入力時に文字列を表示するには input() の () の中に文字列を書く。

```
str = input("文字を入力してください：")
```

なお、input() 関数で入力される値は全て文字列となる。数値を入力する場合にはデータ型変換を行う必要がある。

```
x = int(input("数値を入力してください："))
```

5.2 出力：print()

Python で画面に出力するには print() 関数を使用する。print() の () の中に出力したい文字列や数値を書く。複数の値を出力する場合にはカンマ (,) で区切って並べる。

```
print("abc")      # abc が出力される
print(10)         # 10 が出力される
print(123.45)     # 123.45 が出力される
print("abc",10)   # abc 10 が出力される
```

変数に値を代入し、その値を出力することもできる。また、複数の print() 関数では出力は改行される。改行したくない場合には () 内に end="" を記入する。


```
str = "abc"
print(str)          # abc が出力される
num1 = 10
print(num1)         # 10 が出力される
num2 = 123.45
print(num2)         # 123.45 が出力される
```

```
print("abc", end="")
print(10)           # abc のあと、改行無しで 10 が出力される
```

文字列の中に変数の値を表示するにはいくつかの方法があるが、ここでは f 文字列を使う方法を示す。文字列の前に「f」を書き、文字列の中の置換フィールド { } に直接変数を書く。

```
i = 35
print(f'太郎の年齢は{i}歳です')
                                # 太郎の年齢は 35 歳です と表示される

i=10
j=20
print(f'iの値は{i}で、jの値は{j}です')
                                # iの値は 10 で、jの値は 20 です と表示される
```

置換フィールドの書式を指定すると、書式にフォーマットして出力できる。主な書式を以下に示す。

```
i = 255
print(f'{i:<8d}')      # 8 桁確保して左詰めで出力
print(f'{i:^8d}')      # 8 桁確保して中央に出力
print(f'{i:>8d}')      # 8 桁確保して右詰めで出力
print(f'{i:x}')        # 16 進で出力   ff と出力

f=123.2345
print(f'{f:.2f}')      # 小数点以下 2 桁まで出力
print(f'{f:>10.4f}')   # 10 桁確保して右詰めで小数点以下 4 桁まで出力
```

6 制御文

6.1 分岐処理：if 文

条件分岐処理は if 文でかける。

if 文の条件文の区切りはコロン (:) である。また、ブロックはインデント（字下げ）で示す。

```
if x == 10:
    print("x は 10 です")          # x が 10 の時に実行される
elif x==5:
    print("x は 5 です")          # x が 10 ではなく 5 である時に実行される
else:
    print("x は 10 でも 5 でもありません")  # x が 10 でも 5 でもない時に実行される
```

6.2 繰り返し処理

繰り返し処理は while 文と for 文でかける。

6.2.1 while 文

while 文は while のあとに繰り返しの条件式を書く。条件式はコロン (:) で区切る。この条件式が成立している間、インデントされたブロック内の処理を繰り返し実行する。

```
i=0
while i<10:
    print(i)
    i += 1
```

結果：

```
0
1
2
3
4
5
6
7
```

```
8
9
```

6.2.2 for 文

for 文はリストの各要素、辞書のキー、文字列の各文字などに対しての処理を繰り返す。for のあとに取り出したリストの要素などを代入する変数名を書き、そのあとの in の後ろに取り出す対象となるリストなどを書く。区切りにはコロン (:) を書く。

```
for n in [1,2,3]:          # リスト [1,2,3] の各要素に対する処理を繰り返す
    print(n)              # n にはリストの要素が入り 1,2,3 が表示される

for k in {'key1':1,'key2':2}: # 辞書の各キーを取り出し、繰り返し処理を行う
    print(k)              # k には辞書のキーが入り、key1,key2 が表示される

for c in "abc":           # 文字列の各文字に対する処理を繰り返す
    print(c)              # c には文字列の各文字が入り、a,b,c が表示される
```

なお、for 文で range() 関数を用いることにより、連番や等差数列を生成した処理を行うことができる。range 関数は以下の書き方が可能である。

```
range(stop)                # 0 から stop-1 までの連番、range(5) は 0,1,2,3,4 を生成する
range(start, stop)         # start から stop-1 までの連番、
                           # range(1,5) は 1,2,3,4 を生成する
range(start, stop, step)   # start から stop までの増分 step の等差数列、
                           # range(0,5,2) は 0,2,4 を生成する
```

for 文で使用すると以下にのようになる。

```
for i in range(5):
    print(i)                # 0,1,2,3,4 が表示される

for i in range(1,5):
    print(i)                # 1,2,3,4 が表示される

for i in range(0,5,2):
    print(i)                # 0,2,4 が表示される
```

7 関数：DEF 文

6.2.3 continue 文と break 文

繰り返し処理の中で、処理を中断し次の繰り返し処理に移るには continue 文を、繰り返しを終了させるには break 文を用いる。

```
for i in range(10):
    if i == 3:
        continue;
    if i == 5:
        break;
    print(i)          # 0,1,2,4 が表示される
```

この例では i が 3 の場合には continue 文を実行するため print(i) が実行されない。また、i が 5 になると for 文を抜けている。

break 文を使うと、無限ループの while 文から条件により抜けることができる。

```
while True:
    i = int(input("Please enter integer: "))
    if i == 0:
        break
```

7 関数：def 文

Python では def 文を用いて関数を定義する。下記に 2 つの引数を受け取り、その和を戻り値として返す関数 add(x,y) を示す。関数名の最後はコロン (:) で区切る。

```
def add(x,y):
    z = x + y
    return z
```

関数内はインデントする。戻り値は return 文で指定する。この関数と、関数を呼び出すプログラムを以下に示す。

```
def add(x,y):
    z = x + y
    return z

n = add(2,3)
print(n)          # 慣習的に、関数の最後には空白行を置く
                  # 関数を呼び出すプログラム 戻り値が n に代入される
                  # 5 が表示される
```

8 組み込み関数とモジュール

呼び出し側から関数の渡す値を引数、関数から戻す値を戻り値と呼ぶ。

引数は 0 個を含め複数個かける。引数の間はカンマ (,) で区切る。

Python では戻り値は、なしまたは一つ以上である。戻り値がない場合には return 文を省略可能である。複数の戻り値を返す場合にはカンマ (,) で区切る。

```
def calc(x,y):
    z = x + y
    w = x - y
    return z, w          # 複数の戻り値 z と w をリターンする

n,m = calc(3,2)         # 戻り値が n と m に代入される
print(n,m)              # 5 と 1 が表示される
```

なお、関数は呼び出す前に定義しておく必要がある。以下のプログラムはエラーとなる。

```
n = add(2,3)            # 関数定義前に呼び出しているためエラーとなる

def add(x,y):
    z = x + y
    return z
```

8 組み込み関数とモジュール

プログラム内で関数を定義しなくても、予め Python が用意している関数や、自分が別途作成した関数を使うことができる。そのような関数の集まりをモジュールという。

Python が用意している関数のうち基本的なもの (input() 関数や print() 関数などの組み込み関数) は宣言なしに使用することができるが、基本的なものの以外はプログラム内で宣言する必要がある。宣言の必要な関数は複数の関数を収めたモジュールとして提供されている。

モジュールの使用を宣言するには import 文を使用する。例えば乱数関係の関数を集めた random モジュールの宣言は以下である。

```
import random
```

モジュールをインポートしたら、その中の関数はモジュール名. 関数名 の形で使用できる。乱数モジュールに含まれる randint(引数) 関数 (引数の範囲の乱数を発生させる) は以下のように使用できる。

```
import random

print(random.randint(1,20))
```

なおモジュール内のある関数のみを使用する場合には以下のようにかける。

```
from random import randint

print(randint(1,20))
```

この場合には関数名だけを記述すれば良い。

9 グローバル変数及びローカル変数

Python の変数はグローバル変数とローカル変数に分けられる。関数の中で値を代入した変数はローカル変数、関数の外で値を代入した変数はグローバル変数となる。

ローカル変数は値を代入した関数の中でしか参照できないが、グローバル変数は関数の中でも参照できる。ただし、関数内でグローバル変数に値を代入する場合は global 宣言をする必要がある。

以下はグローバル変数を関数内で参照している例である。

```
def add(y):
    z = x + y          # 関数内ではグローバル変数 x の参照は可能
    return z

x = 10                 # x はグローバル変数
n = add(5)
print(n)               # 15 が出力される
```

以下は関数内でグローバル変数に値を代入しているためエラーとなる。

```
def swap(y):
    z = x               # 関数内ではグローバル変数 x の参照は可能
    x = y               # 関数内でグローバル変数に値を代入するとエラーとなる
    y = z

x = 10                 # x はグローバル変数
swap(5)
print(x)               # 15 が出力される
```

上記を避けるには関数内で global 宣言を行う

```
def swap(y):
    global x          # 変数 x はグローバル変数であることを宣言
    z = x             # 関数内ではグローバル変数 x の参照は可能
    x = y             # グローバル宣言しているので変数 x に値を代入することができる
    y = z

x = 10                # x はグローバル変数
swap(5)
print(x)              # 5 が出力される
```

以上のようにグローバル変数を関数内で使用することは（参照も含めて）なるべく避けることが望ましい。グローバル変数の値を関数に渡すときには引数として渡すのが一般的である。

なお、ローカル変数を関数から参照することはできないが、同じ名前の関数をグローバル変数として使用することはできる。この場合、ローカル変数とは別の変数として扱われる。

```
def add(x,y):
    z = x + y         # z はローカル変数
    return z

z = 13                # この z はグローバル変数、関数内のローカル変数 z とは別の変数
k = z//2
n = add(2,k)
print(n)
```

10 文字列

10.1 文字列の宣言

Python ではダブルクォートまたはシングルクォートで囲んで文字列を表現する。文字列にシングルクォートを含めたい場合には全体をダブルクォートで囲む。また、ダブルクォートを含めたい場合にはシングルクォートで囲む。

```
s1 = "abcdefg"
s2 = 'abcdefg'
s3 = "abcd'efg'hij"
```

```
s4 = 'abcd"efg"hij'
```

10.2 文字列の連結

文字列の連結には + 演算子を用いる。

```
s1 = "abc"  
s2 = 'def'  
print(str1+str2)      # abcdef が表示される
```

10.3 文字列の参照

文字列 `str` のなかの `n` 番目の文字（先頭を 0 番目と数える）は `str[n]` で参照できる。`n` が文字列の長さを越えたときはインデックスエラーとなる。負の値を指定すると、最後から数える。

```
s = "abcdefgh"  
print(s[5])           # f が表示される  
print(s[10])          # インデックスエラーとなる  
print(s[-1])          # h が表示される
```

10.4 文字列の長さ：len()

文字列の長さは `len()` 関数を用いる。

```
s = "abcdefgh"  
print(len(s))         # 8 が表示される
```

10.5 文字列の切り出し

`str[start:stop]` とすると文字列 `str` の `start <= c < stop` の範囲の文字列を切り出すことができる。最初の文字は 0 番目と数える。

```
s = "abcdefgh"  
print(s[2:6])         # cdef が表示される
```

`start` と省略すると最初から、`stop` を省略すると最後までとなる。また、負の値を指定することもできる。


```
s = "abcdefgh"
print(s[2:])      # cdefgh が表示される
print(s[:6])      # abcdef が表示される
print(s[:-2])     # abcdef が表示される
```

10.6 文字列の分割：split()

文字列の分割 split(区切り文字) メソッドを用いる。リターン値はリストとなる。リストについては後述する。以下は空白で分割した例である。

```
s = "012 345 6789"
print(s.split(" ")) # リスト ["012", "345", "6789"] が表示される
```

10.7 文字列の置き換え及び削除：replace()

文字列の置き換えには replace(置き換え前の文字列、置き換え後の文字列) メソッドを用いる。置き換え後の文字列に ("") を指定すると、削除ができる。

```
s = "出張は東京と横浜に行きます"
print(s.replace("横浜", "千葉")) # 出張は東京と千葉に行きます が表示される
print(s.replace("と横浜", ""))   # 出張は東京に行きます が表示される
```

10.8 文字列の検索：find()

文字列の検索には find(検索したい文字列) メソッドを用いる。検索したい文字列が含まれている場合にはそのインデックス (先頭が 0) を、含まれていない場合には -1 を返す。また、後ろから検索したい場合には rfind を用いる。

```
s = "出張は東京と横浜に行きます。東京は寒そうです。"
print(s.find("東京")) # 3 が表示される
print(s.find("京都")) # -1 が表示される
print(s.rfind("東京")) # 14 が表示される
```

11 リスト

リストには複数の値を格納できる。各値のデータ型は異なっても良い。

11.1 リストの宣言

リストは以下のように宣言する。

```
list = [1,2,3,4, "abc"]    # 要素 1,2,3,4, "abc"を持つリスト list を宣言
z = []                    # 空の要素を持つリスト z を宣言
```

11.2 リストの要素の参照及び変更

リストの要素の参照及び変更はインデックス（リストの要素番号）を用いて行う。インデックスは 0 から始まる。

```
list = [1,2,3]
y = list[0]
print(y)                # 1 が表示される
list[2] = 4
print(list)              # [1,2,4] が表示される
```

`list[start:stop]` とすると `list` の `start ≤ c < stop` の範囲の要素を切り出すことができる。最初の要素は 0 番目と数える。`start` を省略すると先頭から、`stop` を省略すると最後までとなる。また、負の値を指定すると後ろから数える。なお、この処理をスライスと呼ぶ。

```
list = ['a','b','c','d','e','f','g','h','i','j','k','l']
print(list[2:6])          # ['c','d','e','f']が表示される
print(list[2:])           # ['c','d','e','f','g','h','i','j','k','l']が表示される
print(list[:6])           # ['a','b','c','d','e','f','g']が表示される
print(list[:-2])          # ['a','b','c','d','e','f','g','h','i','j']が表示される
```

スライスでは、`list[start:atop:step]` の形で、ステップを指定することもできる。

```
list = ['a','b','c','d','e','f','g','h','i','j','k','l']
print(list[2:6:2])        # ['c','e']が表示される
```

11.3 リストの長さ

リストの長さ（要素の個数）は `len()` 関数で取得することができる。

```
list = [1,2,3,4]
print(len(list))          # 4が表示される
```

11.4 リストへの要素の追加

リストの最後に要素を追加するには `append()` メソッドを用いる。

```
list= [1,2,3,4]
list.append(5);
print(list)              # [1,2,3,4,5] が表示される
```

リストに別のリストの要素を追加するには `+` 演算子を用いる。

```
list1= [1,2,3,4]
list2 = [5,6]
print(list1+list2)
```

リストの全ての要素を削除するには `clear()` メソッドを用いる。

```
\begin{lstlisting}
list= [1,2,3,4]
print(list)          # [1,2,3,4]が表示される
list.clear()
print(list)          # []が表示される
```

リストの `n` 番目の要素（最初は 0）を削除するには `pop(n)` メソッドを用いる。 `pop()` の場合は最後の要素を削除する。

```
list = [1,2,3,4]
list.pop(2);
print(list)          # [1,2,4] が表示される
list.pop()
print(list)          # [1,2] が表示される
```

11.5 リストにある要素が含まれるかの確認

リストに指定の要素が含まれるかは `in` 演算子を使用する。

```
list = [1,2,3,4]
```

```
x = 1
if x in list:
    print("含まれています")          # 含まれています と表示
```

11.6 リストの各要素への繰り返し処理

リストの各要素への繰り返し処理には for 文を用いる。

```
list = [1,2,3,4]
for item in list:
    print(item)                      # 1,2,3,4 が表示される
```

12 辞書

辞書にはキーと値のペアを複数格納できる。

12.1 辞書の宣言

辞書は以下のように宣言する。

```
dic = {キー 1: 値 1, キー 2: 値 2, キー 3: 値 3}    # 3つの要素を持つ辞書 dic の宣言
z = {}                                              # 空の要素を持つ辞書 z の宣言
```

辞書は全体を { と } で囲み、キーの後に (:) で値を続ける。複数のキー: 値のペアを (,) で続ける構造である。キーには文字列や数値が使用できる。なお、同じキーを複数回使用することはできない。また、値としては、数値、文字列、リストなどが格納できる。

辞書の例を示す。

```
fruits = {"banana": "円200", "orange": "円100", "apple": "円150"}
tom = {"年齢": 22, "学年": "4年生"}
```

12.2 辞書の要素の参照及び変更

辞書の要素の参照及び変更はキーを用いて行う。

```
fruits = {"banana": "200 円", "orange": "100 円", "apple": "150 円"}
price = fruits["banana"]
```

```
print(price)                                # 200 円 が表示される

fruits["apple"] = "125 円"
print(fruits["apple"])                       # 125 円が表示される
```

12.3 辞書への要素の追加

辞書の要素の追加もキーを用いて行う。

```
fruits = {"banana": "200 円", "orange": "100 円", "apple": "150 円"}
fruits["peach"] = "300 円"
print(fruits)
# {'banana': '200 円', 'orange': '100 円', 'apple': '150 円', 'peach': '300 円'}
# が表示される
```

12.4 辞書の長さ

辞書の長さ（要素の数）は len 関数で取得することができる。

```
fruits = {"banana": "円200", "orange": "円100", "apple": "円150"}
print(len(fruits))                          # 3が表示される
```

12.5 辞書の要素の削除

辞書の要素を削除するには、del 文で削除するキーを指定する。また、pop() メソッドでも削除できる。

```
fruits = {"banana": "200 円", "orange": "100 円", "apple": "150 円"}
del(fruits["banana"])
print(fruits)                               # 'orange': '100 円', 'apple': '150 円' が表示される
fruits.pop("orange")
print(fruits)                               # 'apple': '150 円' が表示される
```

12.6 辞書にキーが含まれるかの確認

辞書に指定のキーが含まれるかは in 演算子を使用する。

```
fruits = {"banana": "円200", "orange": "円100", "apple": "円150"}
print("banana" in fruits)           # True が表示される
print("peach" in fruits)            # False が表示される
```

12.7 辞書の各要素への繰り返し処理

辞書の各要素への繰り返し処理には for 文を用いる。辞書.keys() でキーが、辞書.values() で値が取り出せる。

```
fruits = {"banana": "200 円", "orange": "100 円", "apple": "150 円"}
for key in fruits.keys():
    print(fruits[key])                # 200 円、100 円、150 円が表示される

for value in fruits.values():
    print(value)                      # 200 円、100 円、150 円が表示される
```

また、辞書.items() でキーと値の組み合わせが取り出せる。

```
fruits = {"banana": "200 円", "orange": "100 円", "apple": "150 円"}
for key, value in fruits.items():
    print(key, value) # banana 200 円 orange 100 円 apple 150 円 が表示される
```

13 ファイル入出力

ファイルの入出力の基本的な操作は以下である。

- (1) ファイルのオープン
- (2) ファイルからの読み出し、ファイルへの書き込み
- (3) ファイルのクローズ

13.1 ファイルのオープン及びクローズ

ファイルのオープン時には、ファイル名（パスを含む）、モード（読み込みか書き込みか）、エンコーディングを指定する。

```
f = open("data.txt", mode="r", encoding="utf-8")
# ファイル data.txt を読み込みモードでオープン
```

```
f = open("data.txt", mode="w", encoding="utf-8")  
# ファイル data.txt を書き込みモードでオープン
```

なお、書き込みモードでオープンすると、ファイルが存在しない場合には新規に作成される。

ファイルのクローズは以下のように行う。

```
f = close()
```

with ブロックを使うと、ブロックの終了時にファイルが自動的にクローズされる。

```
with open("data.txt", mode="r", encoding="utf-8") as f:  
# ファイルへの処理
```

13.2 テキストファイルの読み込み

テキストファイルの読み込みには以下の方法がある。

- read(): ファイル全体の読み込み
- readlines(): ファイル全体をリストとして読み込む
- readline(): ファイルを一行ずつ読み込む

ファイル data.txt の中身が以下であるとする（各行の行末に改行コードあり）。

```
I have a pen.  
Do you have an apple?  
I live in Kyoto.
```

これを read() で読み込むと、以下のようになる。

```
f = open("data.txt", mode="r", encoding="utf-8")  
s = f.read()  
print(s)  
f.close()
```

--- 以下のように表示される

```
I have a pen.  
Do you have an apple?  
I live in Kyoto.
```

readlines() で読み込んだ場合を以下に示す。

```
f = open("data.txt", mode="r", encoding="utf-8")
s = f.readlines()
print(s)
f.close()
```

--- 以下のように表示される

```
['I have a pen.\n', 'Do you have an apple?\n', 'I live in Kyoto.\n']
```

readline() を使うと一行ずつ読み込める。

```
f = open("data.txt", mode="r", encoding="utf-8")

while True:
    line = f.readline()
    if line == "":
        break
    print(line, end="")
f.close()
```

--- 以下のように表示される

```
I have a pen.
Do you have an apple?
I live in Kyoto.
```

readline() で一行ずつ読み込んだ場合、最終行まで読み込むと空文字列""が返される。

また、for 文をしてして一行ずつ読み込むことができる。

```
f = open("data.txt", mode="r", encoding="utf-8")

for line in f:
    print(line, end="")
f.close()
```

--- 以下のように表示される

```
I have a pen.
Do you have an apple?
I live in Kyoto.
```

なお、ファイルの各行の行末には改行コード("\n")が含まれている。これを削除するにはrstrip()を用いる。


```
f = open("data.txt", mode="r", encoding="utf-8")

for line in f:
    line = line.rstrip()
    print(line, end="")
f.close()

--- 以下のように表示される
I have a pen.Do you have an apple?I live in Kyoto.
```

13.3 テキストファイルの書き込み

テキストファイルの書き込みには `write()` を用いる。

```
str1 = "I have a pen."
str2 = "Do you have an apple?"
str3 = "I live in Kyoto."

f = open("data.txt", mode="w", encoding="utf-8")
f.write(str1+"\n")
f.write(str2+"\n")
f.write(str3+"\n")
f.close()
```

テキストを一行ずつ改行させたい場合には各業の最後に改行コード ("`\n`") をつけ加える。

以上により、ファイル `data.txt` の中身は以下のようになる。

```
I have a pen.
Do you have an apple?
I live in Kyoto.
```

`data.txt` が存在している場合には内容が上書きされる。また、ファイルが存在していない場合には新規に作成される。

14 練習問題

[練習問題 1] `x` という変数に 2 を代入し、それを 3 倍した数を出力せよ。

[練習問題 2] 変数 `a` に 100 を代入し、変数 `b` に 200 を代入する。その後、両者の値

を入れ替えて、a に 200、b に 100 が代入されているようにせよ。出力として a と b を出力せよ。

[練習問題 3] 変数 a に 10 を代入、変数 b に 2 を代入し、a と b の和、差、積、商を出力せよ。

[練習問題 4] 変数 a に 5 を代入、変数 b に 3 を代入し、a を b で割った時の余りを求めて出力せよ。

[練習問題 5] 変数 a に 5 を代入、変数 b に 10 を代入し、a の b 乗を出力せよ。

[練習問題 6] 変数 a に 5 を代入、変数 b に 10 を代入し、a と b のうち大きい方を出力せよ。

[練習問題 7] 変数 a に 5 を代入し、a が偶数なら Even と、そうでなければ Odd と出力せよ。

[練習問題 8] 変数 a に 8 を代入し、a の 2 乗が 100 以上なら Over と、そうでなければ Under と出力せよ。

[練習問題 9] 変数 n に 100 を代入し、1 から n までの和を求め出力せよ。

[練習問題 10] 1 から n までの和を求めるプログラムを作成し、n を 1 から 10 まで変化させて結果を出力せよ。

[練習問題 11] 変数 n に 10 を代入し、1 から n までの階乗を求めて出力せよ。

[練習問題 12] 1 から n までの階乗を求めるプログラムを作成し、n を 1 から 10 まで変化させ結果を出力せよ。

[練習問題 13] 変数 n に 200 を代入し、1 から n までで 3 で割ると 2 余る数がいくつあるかを求めて出力せよ。

[練習問題 14] 変数 n に 1000 を代入し、1 から n までで 3 で割ると 2 余り、5 で割ると 4 余る数がいくつあるかを求めて出力せよ。

[練習問題 15] 以下のリスト list の各要素の値を 5 倍して表示するプログラムを書け。
list = [2,5,4,5,7,9,10,23]

[練習問題 16] リスト numbers にはいくつかの整数が保持されている。要素を小さい順に並び替えた新しいリスト order を作成せよ。

numbers = [410, 750, 300, 890, 120, 550, 940, 290, 840]

[練習問題 17] 2つのリスト x と y の要素を含む新しいリスト z を作成せよ。ただし、x の要素及び y の要素に重複はある可能性はある。また、z の要素に重複はないとする。

x = [2,4,6,8,10]

y = [1,2,5,6,8]

[練習問題 18] ディクショナリー `scores` には学生の名前とその成績が組みとなって保持されている。 `scores` 内の学生の平均点を求めよ。

```
scores = {"Aoki":75, "Ikeda":80, "Ueda":60, "Emoto":55,
          "Okada":95}
```

[練習問題 19] ディクショナリーのリスト `mean_scores` には 3 回の試験での科目毎の成績が納められている。

```
mean_scores = [ {"英語": 56, "数学": 43, "国語": 63},
                 {"英語": 77, "数学": 58, "国語": 78},
                 {"英語": 35, "数学": 39, "国語": 51}]
```

科目毎の 3 回の試験の平均点を求めよ

[練習問題 20] 1 から 100000 までの乱数を 10 個発生させ、それを 100 で割った数（小数）を小数以下 1 桁まで、一行に 1 つずつ小数点の位置を合わせて表示せよ。なお、各行には「1 個目：」など表示せよ。なお、`a` から `b` までの整数の乱数は以下で求まる。

```
import random
r = randint(a,b)
```

[練習問題 21] 2 点 (x_1, y_1) , (x_2, y_2) の距離を求める関数 `dist(x1, y1, x2, y2)` を定義せよ。 `dist` 関数を用いて、 $(0, 0)$ と $(3, 4)$ の距離、および、 $(2, 3)$ と $(14, 8)$ の距離を求めるプログラムを書け。平方根を求めるには、プログラムの先頭で `math` モジュールを `import` しておかねばならない。`x` の平方根は、`math.sqrt(x)` で求められる。

15 オブジェクト指向プログラミング

本章では Python でのオブジェクト指向プログラミングについて述べる。

15.1 クラス定義とインスタンス生成

オブジェクト指向プログラミングにおけるオブジェクトとは”物”を抽象化した概念で、属性（プロパティ）と動作（メソッド）を持っている。オブジェクトにはクラスとインスタンスがある。

- クラス：オブジェクトのプロパティやメソッドを定義したもの

- インスタンス：クラスに基づき作成したオブジェクトの実体、実際の処理はインスタンスに対して行う

銀行口座を例にクラスとインスタンスを説明する。銀行口座のプロパティとメソッドを以下に示す。

- プロパティ
 - － 銀行口座名
 - － 預金金額
- メソッド
 - － 口座開設
 - － 預け入れ
 - － 引き出し
 - － 残高紹介

銀行口座オブジェクトとして上記のようなプロパティとメソッドがあることを定義するのがクラス定義である。銀行口座オブジェクト（Account）のクラス定義の例を以下に示す。

```
1 class Account:
2
3     def __init__(self, account_name, amount):
4         self.account_name = account_name        # 預金口座名
5         self.amount = amount                    # 預金残高
6
7     def deposit(self, value):                    # 預け入れメソッド
8         self.amount += value
9
10    def draw(self, value):                        # 預金引き出しメソッド
11        ....
12        return value
13
14    def get_balance(self, account_name):          # 残高紹介メソッド
15        return self.amount
```

ここで `__init()` はコンストラクタと呼ばれるメソッドであり、クラスからインスタンスを生成する時に呼ばれ、プロパティの値を初期化する（詳細は後述）。`deposit()`、`draw()`、`get_balance()` は預金預け入れ、預金引き出し、残高紹介のメソッドである。

クラスを使用するにはインスタンスを生成する必要がある。Account クラスのインスタンスを生成するには以下の記述を行う。

```
1 matsui = Account("Matsui", 10000)
```

ここでは”Matsui”という名前の銀行口座インスタンスを最初の預入金 10,000 円で生成している。預け入れ、引き出し、残高照会は以下のように行う。

```
1 class Account:
2     def __init__(self, account_name, amount):
3         self.account_name = account_name
4         self.amount = amount
5
6     def deposit(self, value):
7         self.amount += value
8
9     def draw(self, value):
10        . . . .
11        return value
12
13    def get_balance(self, account_name):
14        return self.amount
15
16 def main():
17     matsui = Account("Matsui", 10000)
18     matsui.deposit(20000)
19     matsui.draw(5000)
20     print(matsui.get_balance())
21
22 if __name__ == "__main__":
23     main()
```

15.2 クラスの継承およびオーバーライド

クラスの継承とはあるクラスの機能（属性やメソッド）を持ち、新たな機能を追加した新しいクラスを作成することである。

以下に名前と年齢の属性を持つ Human クラスと、Human クラスを継承し学生番号の属性を追加した Student クラスの例を示す。

```
1 class Human:
2     def __init__(self, name, age):
3         self.name = name
4         self.age= age
5
6     def get_name(self):
7         return self.name
8
9     def get_age(self):
10        return self.age
11
12    def print_status(self):
13        print("Name:",self.name, "┆Age:", self.age)
14
15 class Student(Human):
16     def __init__(self, name, age, number):
17         super().__init__(name, age)
18         self.number = number
19
20     def get_number(self):
21         return self.number
22
23     def print_status(self):
24         print("Name:",self.name, "┆Age:", self.age, "┆Number:", self.
25             number)
26
27 def main():
28     student1 = Student("Yamamoto", 21, "T2045001")
29     print(student1.get_name())
30     print(student1.get_age())
31     print(student1.get_number())
32     student1.print_status()
33
34 if __name__=="__main__":
35     main()
```

ここで、class Student(Human) は Student クラスは Human クラスを継承していることを示している。また、super().__init__() は親クラスのインストラクタを呼び出している。また、student1.name() のように親クラスのメソッドを呼び出すことができる。

Student クラスの定義で親クラスのメソッドと同じ名前のメソッド `print_status()` を定義し、`number` の出力を追加している。これをオーバーライドと呼ぶ。

演習 【1】

以下を行うプログラム `account.py` を作成せよ。

- (1) 15.1 章の `Account` クラスの `draw()` メソッドを完成させよ
- (2) 複数人の銀行口座を開設し、預け入れ、預金引き出しを繰り返せ
- (3) 銀行口座の名前、残高を表示せよ